Arquitecturas — Guía 8

RCCD 2C 2026 · Resumen de las prácticas de Arquitecturas (estilos y coordinación del 22/10, middleware y organización física del 27/10), la teórica 15 (Cómputo distribuido: Cloud, Fog, Edge) y las notas de Relojes de Lamport y TrueTime. Bibliografía: van Steen & Tanenbaum, *Distributed Systems*, 4ª ed., cap. 2.

0 · La idea en criollo

- Un sistema distribuido es un **montón de programas en máquinas distintas que se hablan por mensajes** y que el usuario ve como uno solo.
- La **arquitectura** responde a dos preguntas:
  - **Lógica (de software):** qué piezas hay (**componentes**), cómo se hablan (**conectores**) y con qué estilo.
  - **Física (de sistema):** en qué máquina corre cada pieza (cliente, servidor, P2P, cloud / fog / edge).
- Casi todos los ejercicios son de **diseño y justificación**: dibujás cajas y flechas, ponés el protocolo en cada flecha y explicás por qué.

> **Receta para cualquier ejercicio de diseño (ej. 1, 2, 3, 4, 10)**
>
> 1. **Componentes** (cajas): listarlos y, para cada uno, su **interfaz** (las operaciones que ofrece), su **procesamiento** y sus **datos**.
> 2. **Conectores** (flechas): etiquetar cada uno con el **mecanismo / protocolo** (HTTP/TLS, SQL, RPC, WebSocket, SMTP, cola, pub/sub).
> 3. **Niveles lógicos**: marcar qué parte es interfaz, cuál procesamiento y cuál datos.
> 4. **Coordinación**: para cada conexión, preguntarse si el emisor tiene que saber quién es el receptor y si el receptor tiene que estar vivo en ese momento. Eso la ubica en la matriz de acoplamiento.
> 5. **Ubicación física** (si la piden): en qué máquina o capa (cliente / servidor / edge / fog / cloud) corre cada componente, y por qué (latencia, volumen de datos, autonomía, regulación).
> 6. Escribir los **supuestos**.

1 · Componentes, conectores, estilos

- **Componente**: unidad modular con **interfaces provistas y requeridas** bien definidas, **reemplazable** dentro de su entorno. Se puede reemplazar, incluso con el sistema andando, **solo si su interfaz no cambia**.
- **Conector**: mecanismo que media la comunicación, coordinación o cooperación entre componentes: llamadas a procedimiento (locales o **remotas**, RPC), pasaje de mensajes, streams de datos.
- **Estilo arquitectónico**: se define por los componentes, cómo se conectan, qué datos intercambian y cómo se configuran juntos. Los principales son **en capas**, **orientado a servicios** y **publish-subscribe**. En la práctica se combinan.
- **Servicio / interfaz / protocolo**:
  - **Servicio**: lo que ofrece una capa (por ejemplo, transporte confiable).
  - **Interfaz**: cómo se accede al servicio (`socket()`, `connect()`, `send()`, `recv()`). Oculta la implementación.
  - **Protocolo**: las reglas del intercambio entre las partes (TCP). Además puede haber un protocolo de aplicación implícito, por ejemplo un eco que agrega un "*".

Tres niveles lógicos de una aplicación

Nivel	Qué hace	Ejemplo (buscador de alquileres, ej. 1b)
Interfaz	Interacción con el usuario o con otra aplicación externa	El navegador de cada PC: 10 instancias del mismo componente
Procesamiento	El núcleo funcional	El sitio web: genera las consultas, las ordena por relevancia (ranking) y arma el HTML
Datos	Opera sobre una base de datos o un sistema de archivos	El almacenamiento distribuido del datacenter

Conectores en ese ejemplo: **HTTP/TLS** (request-reply) entre navegador y sitio, y **SQL o RPC** entre sitio y almacenamiento. Son niveles **lógicos**, no máquinas: los tres podrían correr en una sola.

Si el sitio es **stateless** (no guarda estado entre pedidos), se puede **replicar** y repartir la carga entre réplicas.

2 · Estilos

- **En capas**: cada capa usa los servicios de la de abajo. El problema son las **dependencias fuertes**: el caso de *left-pad*, una dependencia de NPM que al retirarla rompió miles de programas.
- **Orientado a servicios**:
  - **Objetos**: estado + métodos; el conector es una llamada (posiblemente remota); son reemplazables si se mantiene la interfaz.
  - **Microservicios**: cada uno es un proceso de red separado; la clave es la modularización.
  - **Composición de servicios de grano grueso**: servicios grandes, posiblemente de **otras organizaciones** (un sistema de pagos, el correo). La integración no es trivial porque cada uno tiene su propia interfaz.
- **REST** (basado en recursos):
  1. Los recursos se nombran con **URIs**.
  2. Todos ofrecen la **misma interfaz**: `PUT` (modifica), `POST` (crea), `GET` (lee), `DELETE` (borra).
  3. Los mensajes son **autodescriptos**.
  4. La ejecución es **stateless**.
- **Publish-subscribe**: se separa el **procesamiento** (cada proceso es autónomo) de la **coordinación** (cómo se comunican). Los procesos **publican eventos** y se **suscriben** a los que les interesan **sin conocerse entre sí**. Sirve cuando los procesos entran y salen seguido y conviene que las dependencias sean débiles.

3 · Coordinación: la matriz de acoplamiento (ej. 2, 3, 4)

- **Acoplamiento referencial**: para comunicarse, un proceso tiene que **conocer el nombre o identificador** del otro.
- **Acoplamiento temporal**: los dos tienen que estar **andando al mismo tiempo**.

	Temporalmente acoplado	Temporalmente desacoplado
Ref. acoplado	**Directa**: llamada telefónica · RPC · HTTP request-reply · redirect · webhook	**Mailbox**: mail, mensaje de voz, cola de un receptor. El buzón se direcciona explícitamente, pero no hace falta estar activos a la vez.
Ref. desacoplado	**Event-based**: radio en vivo, alarma de incendio, pub/sub que **entrega el dato**. No se conocen, pero el suscriptor tiene que estar vivo cuando se publica.	**Shared data space**: cartelera de avisos, tuple space, cola compartida de trabajos, pub/sub que **almacena**. Lectura por **patrón** (búsqueda asociativa).

Para cada flecha del diseño hay que contestar: **¿el emisor sabe quién es el receptor?** y **¿el receptor tiene que estar vivo en ese momento?**

Ejemplo resuelto en clase: compra de un e-book (ej. 2)

Conexión	Referencial	Temporal	Tipo
Navegador → storage (GET a una URL firmada)	Acoplado (conoce la URI)	Acoplado (request-reply)	Directa
Sitio → correo (SMTP)	Acoplado (servidor y casilla de destino)	Desacoplado (store-and-forward)	Mailbox
Correo → cliente (IMAP / webmail)	Acoplado (su casilla)	Desacoplado	Mailbox
Sitio → pagos (redirect HTTP 302 a través del navegador)	Acoplado	Acoplado	Directa
Pagos → sitio (**webhook**, la confirmación confiable)	Acoplado (URL de notificación)	Acoplado	Directa

Interfaces: sitio (`buscar, agregarCarrito, generarPedido, confirmarPedido`) · storage (`obtenerURLFirmada, descargarLibro`) · correo (`enviar(destino, url)`) · pagos (`iniciarPago(monto, urlRetorno)`). El libro **no pasa por el sitio**: el navegador lo baja directo del storage con una URL firmada que vence.

Pub/sub: arquitecturas y suscripciones

- **Event-based** (event bus) vs **shared data space** (espacio persistente). En los dos, los procesos no tienen referencias explícitas entre sí. Se suelen combinar: te suscribís con un patrón y te notifican cuando alguien inserta una tupla que coincide.
- **Topic-based** = pares **(atributo, valor)**, por ejemplo `(tema, alertas)`. **Content-based** = pares **(atributo, rango)**, por ejemplo `(temperatura, [35, ∞))`. Cuanto más expresiva la suscripción, más caro el **matching** en el middleware.
- **¿La notificación lleva el dato o solo un aviso?**
  - **Con el dato**: el middleware no guarda nada, así que queda **temporalmente acoplado**.
  - **Solo aviso**, y el suscriptor lee después: el middleware **guarda** el dato (con un **lease**, o sea un tiempo de retención), así que desacopla también en el tiempo.

Ejemplo resuelto en clase: simulaciones de 10–15 min con más de 100 clientes (ej. 3)

```
1. Cliente  → API de simulaciones : pedido(parámetros)  → devuelve jobId
2. Cliente  → middleware pub/sub  : suscribir(job.<jobId>.done)
3. API      → cola compartida     : encolar(job)
4. Worker   ← cola                : tomar(job)          (corre 10–15 min)
5. Worker   → almacén de resultados: guardar(resultado)
6. Worker   → middleware          : publicar(job.<jobId>.done, URI)
7. Middleware → cliente           : notificar(URI)
8. Cliente  → almacén             : GET URI
```

	Event-based	Shared data space
Acoplamiento	Ref. desacoplado, temp. acoplado	Desacoplado en los dos ejes
Cliente desconectado cuando termina el job	**Pierde** la notificación	Lee el resultado cuando vuelve
Almacenamiento en el middleware	No	Sí, con leases
Latencia de entrega	Menor	Mayor (hay una lectura posterior)

- Con trabajos de 15 min y más de 100 clientes manda el desacople temporal: **shared data space**, o pub/sub con suscripciones persistentes.
- Se notifica **con la URI**, sin adjuntar el resultado:
  - los resultados son grandes;
  - la descarga se puede reanudar si se corta;
  - la retención se maneja con leases.
- Dimensionar workers: con una ráfaga de 100 pedidos, la última tanda termina a `⌈100/W⌉ × 15 min`. Para que nadie espere más de 30 min, W ≥ 50.

4 · Middleware, wrappers y broker (ej. 5)

- **Middleware**: capa lógica entre las aplicaciones y los SO locales, que se extiende sobre todas las máquinas y ofrece **la misma interfaz** en todas. Es al sistema distribuido lo que el SO a una computadora: comunicación, seguridad, enmascaramiento de fallas, y oculta las diferencias de hardware y SO.
- **Wrapper (adapter)**: le ofrece a una aplicación **la interfaz que ella espera** y traduce cada llamada a las funciones de otro componente. Si cambia la interfaz de un componente, hace falta un wrapper para no tocar a quienes lo usan. Se cuenta **un wrapper por cada par** (quien llama, interfaz que espera) → componente destino.
- **Broker (mediator)**: componente **lógicamente centralizado** por el que pasan todos los accesos. Conoce a todas las aplicaciones y transforma y combina pedidos y respuestas.

N aplicaciones, todas contra todas	N = 2	3	5	20	100
Sin broker: **N(N−1)** (un wrapper por cada par ordenado)	2	6	20	380	9.900
Con broker: **2N** (ida y vuelta de cada aplicación al broker)	4	6	10	40	200

- El broker conviene desde **N = 4**: $N(N-1) > 2N \Leftrightarrow N > 3$.
- A cambio, el broker es un punto central: **cuello de botella**, **punto único de falla** y un **salto extra** en cada llamada.

5 · Organización física: cliente-servidor y tiers

Cliente-servidor (request-reply)

- El **servidor** implementa un servicio; el **cliente** lo pide y espera la respuesta. Un servidor también puede ser cliente de otro.
- Sin conexión (**UDP**) es eficiente si la red es confiable (una LAN). Con conexión (**TCP**) es lo habitual en Internet, pero armar y cerrar la conexión cuesta, sobre todo con mensajes chicos.

¿Y si se pierde un mensaje? Idempotencia (ej. 6)

Supuesto: el cliente espera la respuesta con un timeout y puede retransmitir. Ante un timeout, el cliente **no puede distinguir** entre tres casos:

1. se perdió el pedido;
2. el servidor cayó antes de procesar;
3. se procesó y **se perdió el OK**.

- **Si retransmite**: en el caso 3 la transferencia se ejecuta **dos veces**.
- **Si no retransmite**: en los casos 1 y 2 la transferencia **nunca se ejecuta**, y el cliente no lo sabe.
- El problema de fondo: **transferir no es idempotente**. Una operación idempotente se puede repetir sin daño: "¿cuánto saldo tengo?" sí lo es; "transferir $10.000", no.
- Soluciones:
  - **Identificador de transacción**: el cliente numera cada pedido y el servidor recuerda los que ya atendió. Ante un reintento reenvía la respuesta sin repetir la operación, así que se ejecuta **a lo sumo una vez**.

(¿Hasta cuándo guarda esos números?)
    - **Hacer idempotente la operación**: en lugar de "transferir $100", "crear la transferencia #7 por $100". Si llega dos veces, la segunda no hace nada.
- **Con TCP no alcanza**: si la conexión se corta después de transferir y antes del OK, vuelve la misma ambigüedad. **Transporte confiable ≠ operación confiable.**

Dos tiers físicos: ¿dónde se corta?

Las organizaciones (a)–(e) reparten interfaz / aplicación / datos entre la máquina cliente y la servidora. Si una capa aparece en las dos máquinas, está **partida** entre ellas:

Org.	En el cliente	En el servidor	Ejemplo
(a)	parte de la interfaz	resto de la interfaz + aplicación + datos	Terminal; el servidor controla la presentación
(b)	toda la interfaz	aplicación + datos	Navegador con HTML
(c)	interfaz + parte de la aplicación	resto de la aplicación + datos	El cliente valida un formulario antes de mandarlo
(d)	interfaz + aplicación	datos	App bancaria que arma las transacciones en el cliente y las sube
(e)	interfaz + aplicación + parte de los datos (caché local)	datos	Trabajo offline con sincronización

- **Clientes livianos (thin), (a)–(c)**: fáciles de administrar, poco software en el usuario y poca dependencia de su plataforma, pero interfaz menos sofisticada.
- **Clientes pesados (fat), (d)–(e)**: suites de oficina, multimedia, trabajo sin conexión.

Ejemplo resuelto en clase: validar un formulario de 20 campos (ej. 7.1)

	En el cliente	En el servidor
Ventajas	Respuesta inmediata; menos pedidos y menos carga en el servidor	**No se puede saltear**; una sola implementación, actualizada en un solo lugar
Desventajas	**Se puede saltear** editando el JavaScript; hay campos que no se pueden verificar en el cliente (unicidad, datos de la base)	Cada error cuesta un viaje de ida y vuelta; más carga en el servidor

Conviene validar **en los dos lados**: en el cliente por usabilidad y en el servidor por seguridad.

Tres tiers (multi-tiered)

- **Cliente → servidor de aplicación → servidor de base de datos**. El servidor de aplicación es a su vez **cliente** del de datos. Por ejemplo: un servidor web, detrás un servidor de aplicación, y detrás la base.
- **Distribución vertical**: componentes lógicamente **distintos** en máquinas distintas. Es lo que hacen las arquitecturas multi-tier.
- **Distribución horizontal**: un cliente o servidor partido en partes **equivalentes**, cada una con su porción de los datos (por ejemplo, servidores para A–H, I–P y Q–Z). Reparte la carga. **P2P** es la distribución horizontal llevada al extremo.

6 · Peer-to-peer y overlays (ej. 8, 9)

- **P2P**: todos los procesos son iguales; cada uno es cliente y servidor a la vez.
- **Overlay network**: los nodos son **procesos** y los enlaces son **canales lógicos** (a menudo conexiones TCP). Cada nodo solo le habla a sus vecinos. **No es la red física**: son procesos y canales, no cables ni radios.

Puede ser estructurada o no estructurada.

Estructurado: tablas de hash distribuidas (DHT)

- Topología **determinística** (anillo, árbol, grilla).
- Cada dato tiene una clave, `clave = hash(dato)`. Cada nodo tiene un id del mismo espacio y guarda un rango de claves.
- La operación básica es `nodo = lookup(clave)`: el pedido se rutea por la topología.
- **Chord** (anillo):
  - La clave k la guarda `succ(k)`, el primer nodo con id ≥ k.
  - Cada nodo tiene **atajos** (finger table) a otros nodos. Un lookup avanza lo más lejos posible sin pasarse, así que tarda **O(log N)** saltos.
  - Al entrar o salir, un nodo solo intercambia claves con su sucesor.

No estructurado: hay que buscar

- Cada nodo tiene una lista **ad hoc** de vecinos, así que la overlay es un grafo aleatorio. Al entrar, el nodo pide una lista inicial a un nodo conocido.
- **Flooding**: el pedido va a **todos** los vecinos. Quien ya lo vio lo ignora; quien tiene el dato responde y, si no, lo reenvía. Un **TTL** limita los saltos. Es rápido pero genera mucho tráfico.
- **Random walk**: el pedido va a **un** vecino al azar, que hace lo mismo. Genera mucho menos tráfico, pero tarda más. Con n caminatas en paralelo, el tiempo baja unas n veces.
- Quien tiene el dato responde con un **aviso corto** (qué tiene, su IP y su puerto), directo o por el camino inverso. Después el solicitante abre una **conexión directa** y baja el dato, que **no pasa por la overlay**.

> **Fórmulas de búsqueda**. N nodos, cada dato replicado en r nodos al azar, d vecinos por nodo.
>
> - **Random walk**: $P[\text{encontrar en el intento } k] = (r/N)(1 - r/N)^{k-1}$. En promedio se consultan $S \approx N/r$ nodos, uno detrás de otro.
> - **Flooding**: después de k pasos alcanza a lo sumo $R(k) = d(d-1)^{k-1}$ nodos. Lo encuentra cuando $(r/N) \cdot R(k) \geq 1$.
>
> Ejemplo: N = 100.000, r = 100, d = 10. Random walk necesita S ≈ 1000 nodos. Flooding necesita R(k) ≥ 1000: con R(3) = 810 no alcanza y con R(4) = 7290 sí, así que son 4 pasos y unos 7290 nodos tocados.

Ejemplo resuelto en clase: distribuir un tar.gz de 1 TB (ej. 9)

Ojo: la clase usó una versión con **100 ms de propagación y 1 Gbps**; la guía dice **200 ms de RTT**. Lo que importa es el modelo.

- **Datos**:
  - `1 TB / 100 MB = 10.000 piezas`.
  - `100 servidores × 1000 piezas = 100.000` piezas guardadas, así que cada pieza está en **10 servidores** (r/N = 10%).
  - Transmitir una pieza: `800 Mb / 1 Gbps = 0,8 s`.
- **Modelo** (escribir los supuestos):
  1. El pedido recorre h saltos por la overlay y el aviso vuelve por el mismo camino.
  2. Después, el nodo pide la pieza directo al servidor que la tiene.
  3. Las piezas se bajan de a una.

  $T_{pieza} = (2h + 2) \cdot 100 \text{ ms} + 0{,}8 \text{ s}$.
- **Flooding** (d = 10):
  - Paso 1: R(1) = 10, así que en promedio 1 vecino tiene la pieza. Ninguno la tiene con probabilidad $0{,}9^{10} \approx 0{,}35$.
  - Paso 2: casi seguro la encuentra.

- $E[h] = 0{,}65 \cdot 1 + 0{,}35 \cdot 2 = 1{,}35$.

- **Random walk**: $h \approx N/r = 10$.
- **Super peer / broker**: $h = 1$, porque sabe quién tiene cada pieza.

Estrategia	h	Mensajes por pieza	Por pieza	Total (10.000 piezas)
Flooding (TTL 2)	1,35	4,7	≈ 1,27 s	≈ 3,5 h
Random walk	≈ 10	≈ 22	≈ 3,0 s	≈ 8,3 h (**no baja: sube**)
Super peer	1	4	≈ 1,2 s	≈ 3,3 h (a cambio, punto central)

P2P jerárquico: super peers

- Son nodos especiales que mantienen un **índice** de los datos o hacen de broker. Cada **weak peer** se conecta como cliente a un super peer y todo su tráfico pasa por él.
- Tienen que ser procesos estables y muy disponibles; se pueden emparejar para tener respaldo.
- Decidir quién es super peer es un problema de **elección de líder**.

Ruteo por vecinos vs anillo (ej. 8.3)

- **Flooding / random walk**: no requieren mantener estructura y toleran que la topología cambie (nodos que entran y salen). Como contra, flooding genera mucho tráfico y random walk tiene mucha latencia; ninguno garantiza encontrar el dato si el TTL se agota.
- **Anillo** (estructurado): el lookup es determinístico, con O(N) saltos en un anillo simple u O(log N) con atajos tipo Chord. Como contra, hay que mantener la estructura cuando los nodos entran o salen, y un nodo caído corta el anillo si no hay atajos ni sucesores de respaldo.

7 · Cloud, fog y edge (ej. 10)

- **Cloud computing**: pool de recursos **virtualizados** en datacenters, configurable dinámicamente (más trabajo, más recursos) y pago por uso. Se ofrece en tres modelos:
  - **IaaS**: VMs y almacenamiento.
  - **PaaS**: una API para subir y correr programas.
  - **SaaS**: aplicaciones como Gmail o Docs.

  Como arquitectura es un **cliente-servidor avanzado**: el cliente no sabe dónde está el servidor, que suele estar repartido en todo un datacenter.

Capa	Dónde	Latencia típica	Recursos
Dispositivos	Sensores, autos, celulares	< 1 ms	Muy escasos
Edge	Borde de la red de la organización: servidores propios (on-premise), por ejemplo un campus o un aeropuerto; edge de acceso	1–5 ms	Escasos y heterogéneos
Fog	Infraestructura intermedia: datacenters regionales compartidos	5–20 ms	Medios
Cloud	Datacenters del proveedor, por Internet	50–150 ms	"Infinitos"

Más cerca da menos latencia; más lejos da más recursos. Los límites entre capas son difusos.

¿Por qué no ir directo a la nube?

1. **Latencia**: unos 100 ms a la nube es inaceptable para coordinar un auto autónomo. Es el argumento más fuerte. La luz en fibra recorre unos 200.000 km/s, o sea que 1 ms de ida y vuelta equivale a unos 100 km.
2. **Ancho de banda / volumen**: 1000 cámaras × 4 Mb/s = 4 Gb/s que no conviene subir. El video necesita la fuente cerca (menos jitter).

3. **Confiabilidad / autonomía**: hospitales y fábricas tienen que seguir operando aunque se corte el enlace.
4. **Regulación / soberanía**: hay datos que no pueden salir, como las historias clínicas.

- **Ejemplo de autos en un cruce**: cada auto reacciona en tiempo real con sus propios sensores. Para detectarse más allá de lo que ven, le pasan su posición a una **infraestructura edge local**, que les avisa entre sí. Coordinarse vía la nube no es aceptable.
- **Orquestación** (qué va dónde):
  - **Asignación de recursos** (CPU, memoria, red).
  - **Ubicación de servicios** (cuándo y dónde instalar cada uno; con apps móviles, el edge más cercano).
  - **Selección del edge** (el más cercano no siempre es el mejor).
  - El problema de placement es NP-difícil.

  Criterio: cada servicio va en la capa que cumple con la latencia que tolera y que ve la parte del sistema que necesita.
- Problemas nuevos del continuo:
  - **Migración con movilidad**: el estado sigue al usuario.
  - **Consistencia entre niveles**: fuerte implica latencia de nube; eventual implica conflictos.
  - **Autonomía ante particiones**: CRDTs y relojes híbridos.

> **Plantilla para el ej. 10** (aeropuerto). Para cada componente (vehículo, AP, datacenter, servidores en Internet):
>
> 1. ¿En qué capa va (dispositivo / edge / fog / cloud)?
> 2. ¿Qué **latencia** tolera su función?
> 3. ¿Qué **parte del sistema** necesita ver (un auto, la zona de un AP, todo el predio, varios aeropuertos)?
> 4. ¿Qué **procesamiento y datos** tiene? ¿Tiempo real o histórico?
> 5. ¿Qué **interfaz** ofrece y con qué conector (pub/sub de posiciones, request-reply, streams)?
>
> Y decir cómo se comunican los autos entre sí: directo, o vía el edge que ve la zona.

8 · Cómputo distribuido: límites (teórica 15)

- **Definiciones**:
  - Lamport: "aquel en el que la falla de una computadora que ni sabías que existía puede dejar inutilizable la tuya".
  - Tanenbaum: "colección de computadoras independientes que se presenta como un único sistema coherente".
- **Tres rasgos** (Coulouris): **concurrencia**, **sin reloj global** y **fallas independientes**. La comunicación es **solo por mensajes**: lo que un nodo sabe de otro es lo que le llegó, y puede estar desactualizado.
- **Por qué distribuir**: escalar, tolerar fallas, localidad y costo. Escalar vertical tiene techo; escalar horizontal no, pero exige coordinar.
- **Amdahl**: `S(n) = 1 / ((1−p) + p/n)`. Con $n \to \infty$, $S \to 1/(1-p)$. Con $p = 0{,}95$, por más máquinas que agregues, la aceleración máxima es 20× ($S(1024) \approx 19{,}6$). Gustafson: si el problema crece con n, el panorama mejora.
- **Modelos de sistema**:
  - Tiempo: síncrono / parcialmente síncrono / asíncrono.
  - Fallas: crash / crash-recovery / bizantina.
  - Canales: confiables / con pérdidas equitativas / arbitrarios.

  Todo algoritmo es correcto **respecto de un modelo**.
- **8 falacias** (Deutsch): la red es confiable · la latencia es cero · el ancho de banda es infinito · la red es segura · la topología no cambia · hay un solo administrador · el transporte cuesta cero · la red es homogénea.
- **Imposibilidad**:

  - **Dos Generales**: con un canal con pérdidas no hay acuerdo garantizado, porque siempre falta el último ACK.
  - **FLP**: en un sistema asíncrono no hay consenso determinista que siempre termine si puede caer un proceso.
  - **CAP**: ante una **partición** hay que elegir entre **C**onsistencia (linealizable) y **A**vailability.
  - **PACELC**: si hay Partición, A o C; si no (Else), igual se elige entre **L**atencia y **C**onsistencia.
- **Replicación**:
  - **Máquina de estados replicada**: mismos comandos en el mismo orden dan el mismo estado, así que todo se reduce a acordar el orden del log (consenso).
  - Réplicas necesarias: crash `n ≥ 2f+1` (Paxos, Raft); bizantina `n ≥ 3f+1` (PBFT).
  - **Quórums**: `R + W > N`, así que toda lectura ve la última escritura (N=3, W=2, R=2 tolera una caída).
- **Consistencia**, de más fuerte a más débil:
  - **Linealizable**: cada operación parece instantánea (Spanner, etcd).
  - **Secuencial**: un orden total común.
  - **Causal**: si a pudo influir en b, todos ven a antes que b (chats, redes sociales).
  - **Eventual**: si cesan las escrituras, las réplicas convergen (**DNS**, Dynamo, CRDTs).

  Cuanto más débil, menos coordinación y latencia y más disponibilidad, pero más trabajo para el programador.
- **La nube como sistema distribuido**:
  - Las fallas son la norma: 10.000 servidores con MTBF de 30 años dan ≈ 0,9 fallas por día.
  - **Latencia de cola**: con fan-out a 100 servidores lentos el 1% de las veces, el 63% de los pedidos son lentos ($1 - 0{,}99^{100}$).
  - MapReduce lleva el cómputo a los datos.
- **Péndulo** centralizar / distribuir: mainframe → cliente-servidor → clusters/grid → nube pública (2006) → edge/fog → continuo. Lo mueve la relación entre el costo de computar, el costo de comunicar y la latencia tolerable.

9 · Relojes: Lamport, vectoriales, TrueTime

Lamport (1978): ordenar sin reloj común

- **"Ocurrió antes" (a → b)** se da en tres casos: a y b están en el mismo proceso y a va primero; a es el envío y b la recepción del mismo mensaje; o por transitividad.
- Un contador entero C por proceso, con tres reglas:
  1. Antes de cada evento local o envío: `C := C + 1`.
  2. Al enviar, el mensaje lleva `t = C`.
  3. Al recibir con marca t: `C := max(C, t) + 1`.
- El **max** asegura que la recepción quede después del envío aunque el receptor viniera atrasado.
- **Garantía**: `a → b ⇒ C(a) < C(b)`. **La recíproca no vale**: C(c) = 1 < C(b) = 2 y sin embargo c y b son **concurrentes** (ninguno pudo influir en el otro).
- **Orden total**: desempatar por id de proceso, usando el par (C, p).

```
Ejemplo de clase:  p1: a(1) [envía m1 a p2] → b(2) [envía m2 a p3]
                   p2: c(1) → d(2) → e [recibe m1]
                   p3: f [recibe m2]
e recibe m1 (t=1) con C=2:  max(2,1)+1 = 3   (gana el reloj local)
f recibe m2 (t=2) con C=0:  max(0,2)+1 = 3   (gana la marca del mensaje)
```

Relojes vectoriales (Fidge, Mattern, 1988)

- Cada proceso lleva un vector con un contador por proceso. Al recibir, toma el **máximo componente a componente** y suma 1 en la propia.
- `V(a) < V(b)` (todas las componentes ≤ y alguna <) si y solo si a → b. Si no son comparables, son **concurrentes**. Por ejemplo, c = (0,1,0) y b = (2,0,0).
- **Sí** caracterizan la causalidad, a diferencia de Lamport. El costo es mandar n enteros por mensaje.

Evento	a	b	c	d	e	f
Lamport	1	2	1	2	3	3
Vectorial	(1,0,0)	(2,0,0)	(0,1,0)	(0,2,0)	(1,3,0)	(2,0,1)

Relojes físicos y TrueTime (Spanner, Google, 2012)

- Los relojes físicos **derivan**: un cristal de 20 ppm se desvía unos 1,7 s por día. **NTP** los acerca a milisegundos en Internet y **PTP** a microsegundos en una LAN, nunca a cero.
- **TrueTime**: el reloj **admite que no sabe** la hora exacta y devuelve un **intervalo**:
  - `TT.now() = [earliest, latest]`, de ancho 2ε, que garantiza contener la hora real.
  - `TT.after(t)` es verdadero si t ya pasó con seguridad (t < earliest).
  - `TT.before(t)` es verdadero si t todavía no llegó (t > latest).
- Se sincroniza cada 30 s contra time masters con **GPS y relojes atómicos**. ε arranca en ~1 ms y crece con la deriva supuesta (200 µs/s, diez veces peor que lo real, por seguridad): $\varepsilon = 1\ ms + 0{,}2\ ms/s \times \Delta t$, del orden de 1 a 7 ms.
- **Commit wait**:
  1. Al hacer commit se elige el timestamp `s = TT.now().latest`.
  2. No se liberan los locks ni se responde hasta que `TT.after(s)` sea verdadero, lo que lleva unos **2ε**.
  3. Así, s ya pasó en tiempo real para cualquier observador.
- **Consistencia externa**: si T2 empieza después de que T1 terminó en tiempo real, aunque sea en otro nodo del mundo y solo los conecte un cliente externo, entonces $s_1 < s_2$. **La incertidumbre del reloj se paga en latencia** (unos ms por commit, en parte superpuestos con la ronda de Paxos).
- **Lamport vs TrueTime**: Lamport ordena de forma compatible con la causalidad que **viaja en mensajes**. TrueTime ordena también eventos **sin mensajes entre sí**, porque ancla los timestamps a la hora real.

10 · Mapa de la guía

Ej.	Tema	Mirá
1	Diseñar 3 escenarios: componentes, conectores, protocolos, niveles (docs compartidos, buscador, transferencia entre apps)	§0 receta, §1 (1b resuelto), §3
2	E-book: interfaces, niveles y acoplamientos	§3 (resuelto)
3	Simulaciones con pub/sub; event-based vs shared space; notificaciones	§3 (resuelto)
4	Edificio inteligente: sensores, luces, suscripciones atributo-valor / atributo-rango	§3 topic vs content-based
5	Wrappers vs broker	§4 (5b y 5c resueltos)
6	Transferencias sobre UDP: se pierde el OK	§5 idempotencia (resuelto)
7	Validaciones en el cliente o el servidor; multi-tier	§5 (7.1 resuelto)
8	Sensores WiFi + Bluetooth: overlay, recolección, flooding/RW vs anillo	§6
9	tar.gz de 1 TB: flooding, random walk, super peer	§6 (resuelto con la versión de 1 Gbps)
10	Aeropuerto con vehículos autónomos: cloud / fog / edge	§7 plantilla
—	Teórica sin guía: Lamport, vectoriales, TrueTime, CAP, quórums	§8, §9